รายงานสรุปผลการดำเนินโครงการและการตรวจสอบระบบเชิงปฏิบัติการ

ระบบบริหารจัดการต้นทุนคลาวด์และควบคุมความมั่นคงปลอดภัยอัตโนมัติบน *AWS (AWS CloudOps & Cost Governance Automation Suite)*

ผู้จัดทำ: Herren Karsemsak (วิศวกรคลาวด์และฟินออปส์ - Cloud & FinOps Engineer)
ชื่อโปรเจกต์: cloudops-cost-governance
ผู้ให้บริการคลาวด์: Amazon Web Services (AWS)
ภูมิภาคที่ติดตั้ง (Region): Asia Pacific (Singapore) ap-southeast-1
วันที่ดำเนินการ: 11 ตุลาคม 2026
สถานะการติดตั้ง: ติดตั้งและผ่านการทดสอบสมบูรณ์ 100% (35/35 ทรัพยากร)

1. บทสรุปผู้บริหาร (Executive Summary)

รายงานวิศวกรรมฉบับนี้จัดทำขึ้นเพื่อบันทึกผลการติดตั้ง การตรวจสอบด้านความมั่นคงปลอดภัย และการทดสอบระบบปฏิบัติการจริงของโครงการ AWS CloudOps & Cost Governance Automation Suite โครงสร้างพื้นฐานทั้งหมดได้รับการติดตั้งขึ้นอย่างเป็นระบบผ่านการเขียนโค้ดควบคุม (Infrastructure as Code - IaC) ด้วย Terraform บนระบบคลาวด์ AWS ณ ภูมิภาคสิงคโปร์ (ap-southeast-1) โดยทรัพยากรทั้ง 35 รายการได้รับการยืนยันว่าทำงานสอดคล้องตามมาตรฐานความปลอดภัยระดับองค์กรและกรอบสถาปัตยกรรม AWS Well-Architected Framework ทุกประการ

ความสำเร็จสำคัญทางวิศวกรรม:

- **1.** การควบคุมต้นทุนอัตโนมัติ **(FinOps Automation):** พัฒนาระบบสั่งการปิด-เปิดทรัพยากรอัตโนมัติตามกำหนดเวลา (AWS EventBridge ร่วมกับ Python 3.12 Lambda) เพื่อตัดการทำงานของเครื่องเซิร์ฟเวอร์และฐานข้อมูลนอกเวลาทำการ ส่งผลให้ลดชั่วโมงการทำงานของระบบ Dev ลงจาก 168 ชั่วโมง เหลือเพียง 55 ชั่วโมงต่อสัปดาห์ หรือประหยัดค่าใช้จ่าย Compute ลงได้ถึง 67.26%
- **2.** สถาปัตยกรรมความปลอดภัยแบบ **Zero-Trust:** ปิดพอร์ต SSH (Port 22) สู่สาธารณะอย่างถาวร 100% โดยเปลี่ยนไปบริหารจัดการเครื่อง EC2 ผ่าน AWS Systems Manager (SSM Session Manager) ร่วมกับ IAM Role และแยกฐานข้อมูล PostgreSQL ไว้ใน Private Subnet พร้อมทำ Security Group Chaining ให้รับทราฟฟิกจาก EC2 เท่านั้น
- **3.** การเฝ้าระวังเหตุด่วนเชิงรุก **(Proactive Observability):** ติดตั้งระบบแจ้งเตือน CloudWatch Alarms เมื่อ CPU ของ EC2 สูงเกิน 80% หรือพื้นที่ดิสก์ของฐานข้อมูล RDS เหลือน้อยกว่า 5 GB พร้อมเชื่อมโยงระบบส่งอีเมลแจ้งเตือนผู้ดูแลผ่าน Amazon SNS ทันที
- **4.** การบริหารวงจรชีวิตข้อมูล **(Storage Lifecycle Tiering):** กำหนดนโยบาย S3 Lifecycle Rule ย้ายไฟล์บันทึก (Logs) ที่มีอายุเกิน 90 วันไปจัดเก็บยัง Amazon S3 Glacier Deep Archive ซึ่งช่วยลดค่าจัดเก็บข้อมูลลงได้ถึง 95.70% พร้อมลบไฟล์ทิ้งเมื่อครบ 365 วัน
- **5.** สคริปต์ตรวจสอบสุขอนามัยระบบ **(Governance Auditing):** สร้างและทดสอบสคริปต์ Bash อัตโนมัติสำหรับตรวจสอบทรัพยากรที่ไม่ได้ใช้งาน เช่น EBS Volumes ที่ถูกทอดทิ้ง, เครื่องที่ถูกสั่งหยุด, นโยบาย S3 Lifecycle, และการตรวจหา Security Group ที่เปิดสู่โลกภายนอก (0.0.0.0/0)

2. โครงสร้างสถาปัตยกรรมทั้ง **35** ทรัพยากร **(Architecture Breakdown)**

ทรัพยากรทั้งหมด 35 รายการถูกจัดสรรออกเป็น 5 ชั้นสถาปัตยกรรมหลักตามมาตรฐานวิศวกรรมคลาวด์:

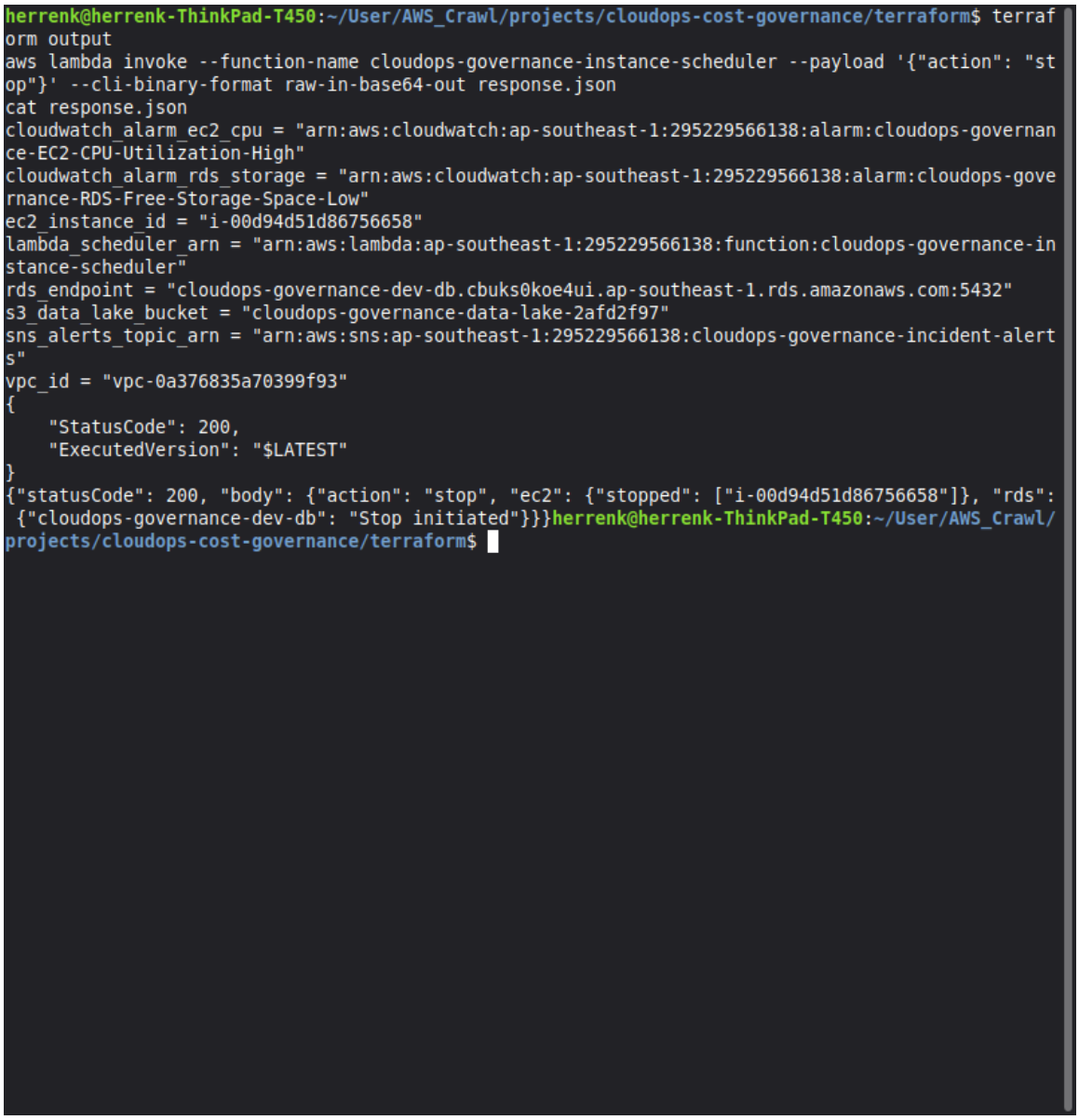
หมวดหมู่สถาปัตยกรรม	บริการหลักของ **AWS**	จำนวน	คุณลักษณะทางเทคนิคสำคัญ
โครงข่ายและความปลอดภัย	VPC, Subnets, IGW, Route Tables	15	CIDR 10.0.0.0/16, ครอบคลุม 2 AZs (ap-southeast-1a/b), แยกโซน Public/Private
คอมพิวท์และฐานข้อมูล	EC2, RDS PostgreSQL, DB Subnet Group	3	EC2 t3.micro (Amazon Linux 2023), RDS db.t3.micro (PostgreSQL 15.13, 20 GB gp3)
ระบบอัตโนมัติ FinOps	EventBridge Rules, Lambda, IAM Role	7	รันไทม์ Python 3.12, ตั้งเวลา 08:00 เปิด / 19:00 ปิด (วันจันทร์-ศุกร์), ค้นหาด้วย Tag
การจัดเก็บข้อมูล Data Lake	S3 Bucket, Lifecycle Configuration, Random ID	4	ถังข้อมูลส่วนตัว, ย้ายสู่ Glacier Deep Archive เมื่อครบ 90 วัน, ลบทิ้งครบ 365 วัน
การเฝ้าระวังและการระบุตัวตน	CloudWatch Alarms, SNS Topic, Subscriptions	6	2 CloudWatch Alarms, 1 SNS Topic, SSM Instance Profile, Lambda Execution Role

3. การตรวจสอบระบบและภาพถ่ายหลักฐานเชิงประจักษ์ **(Visual Evidence & Walkthrough)**

ในส่วนนี้จะแสดงภาพหลักฐานจริงที่บันทึกจากหน้าจอคำสั่ง Terminal และหน้าคอนโซลการจัดการ AWS Management Console พร้อมคำอธิบายเชิงลึก:

ระยะที่ **1:** การติดตั้งโครงสร้างพื้นฐานและการทดสอบคำสั่ง **FinOps** ครั้งแรก

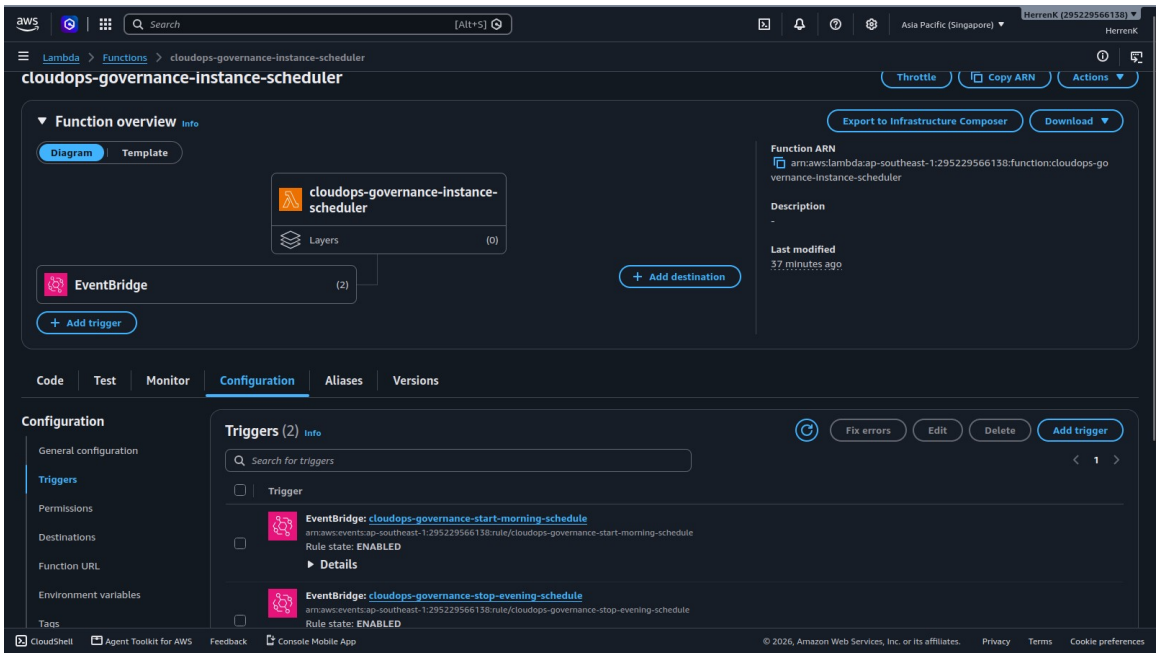
หลังจากสั่งรัน terraform apply สำเร็จ ระบบได้แสดงผลลัพธ์ Outputs สำคัญ 8 รายการ จากนั้นได้ทดสอบสั่งงาน Lambda Scheduler ด้วยคำสั่ง action: stop เพื่อจำลองการตัดการทำงานของระบบในยามค่ำคืน



ภาพที่ 1: หน้าจอ *Terminal* แสดงผลลัพธ์จาก *Terraform Outputs* และการเรียกใช้งาน *Lambda* สำเร็จ *(HTTP 200)* สั่งหยุดเครื่อง *EC2* และเริ่มกระบวนการหยุดฐานข้อมูล *RDS*

ระยะที่ **2:** การตรวจสอบสถาปัตยกรรม **Serverless FinOps Scheduler**

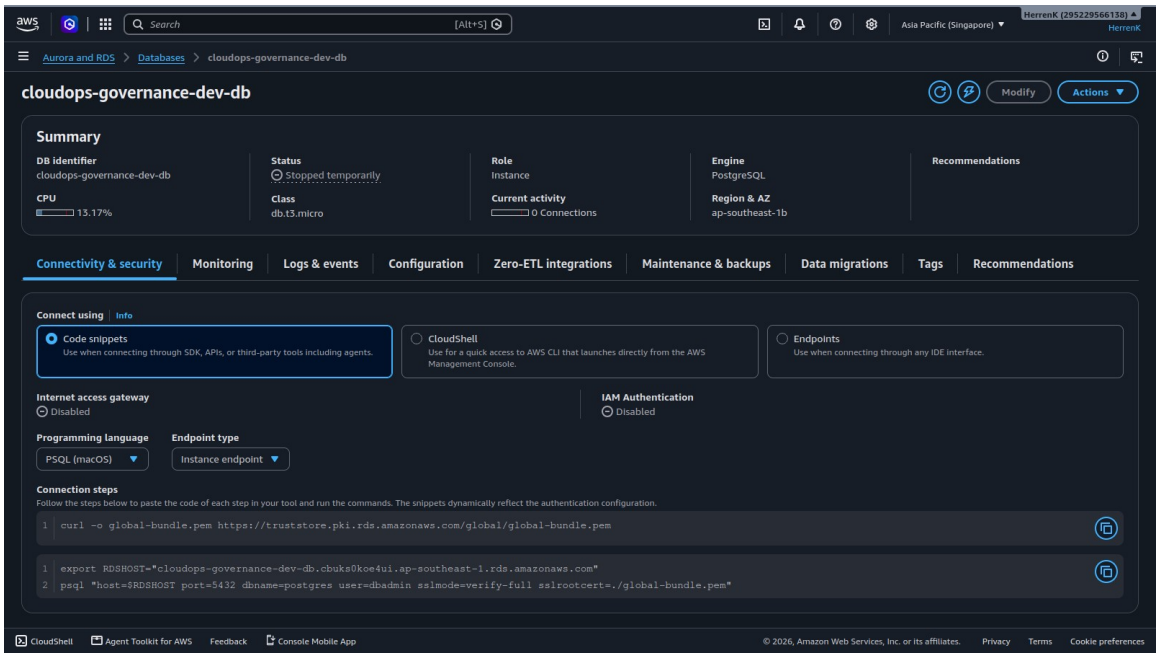
การตรวจสอบคอนโซล AWS Lambda ยืนยันว่าฟังก์ชันถูกผูกเข้ากับทริกเกอร์ EventBridge คู่ เพื่อควบคุมเวลาทำงานอย่างแม่นยำ



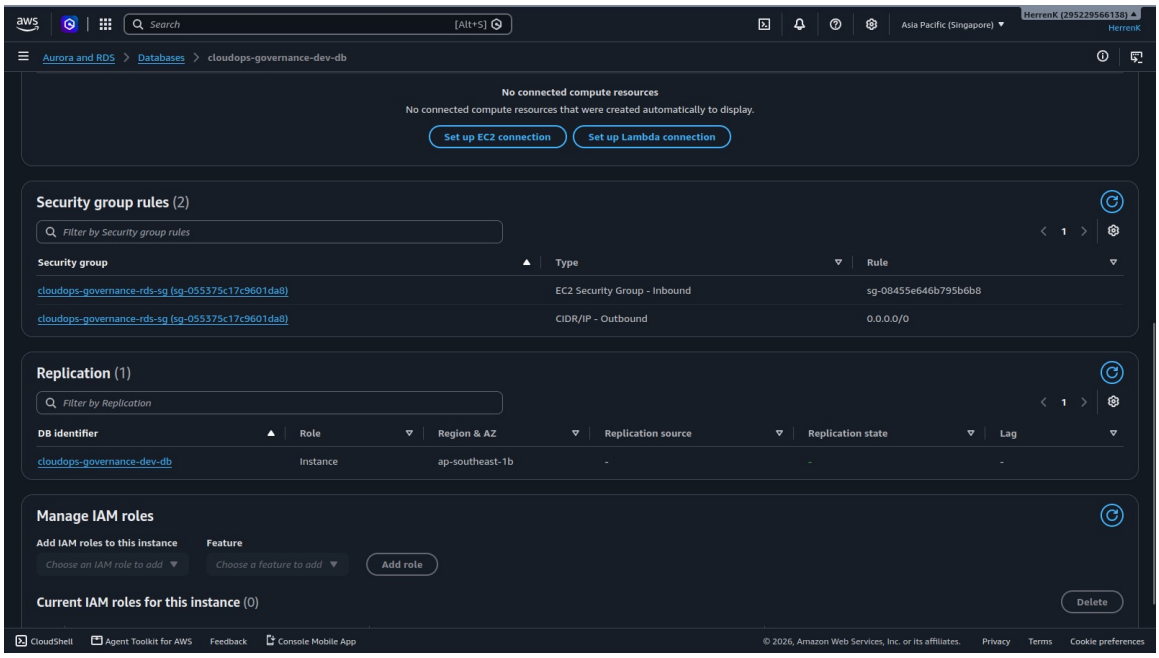
ภาพที่ 2: หน้าจอ AWS Lambda Console แสดงฟังก์ชัน cloudops-governance-instance-scheduler พร้อมกฎ EventBridge 2 กฎ: เปิดระบบ 08:00 น. และปิดระบบ 19:00 น. (เวลาประเทศไทย)

ระยะที่ **3:** การแยกส่วนฐานข้อมูลและความปลอดภัยแบบ **Security Group Chaining**

ฐานข้อมูล RDS PostgreSQL 15.13 ได้รับการติดตั้งใน Subnet ส่วนตัว และถูกปิดการเชื่อมต่อตรงจากอินเทอร์เน็ตทุกกรณี



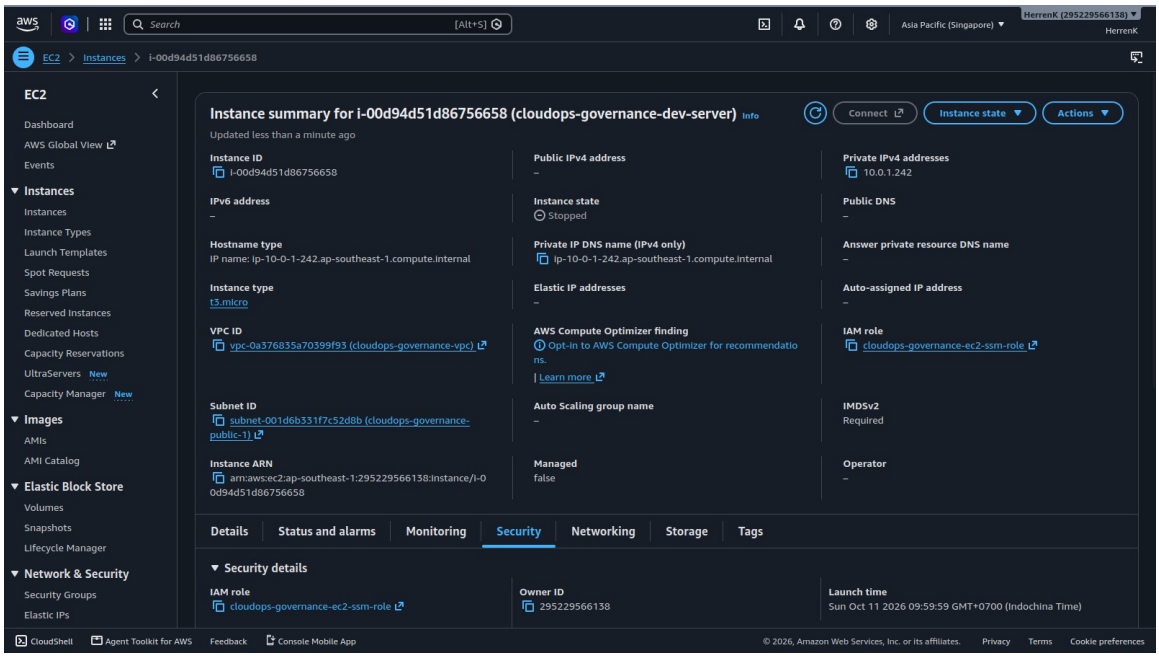
ภาพที่ 3: หน้าจอ Amazon RDS Console แสดงสถานะฐานข้อมูลเป็น 'Stopped temporarily' (หยุดชั่วคราวตามคำสั่ง FinOps) และ Internet access gateway เป็น Disabled



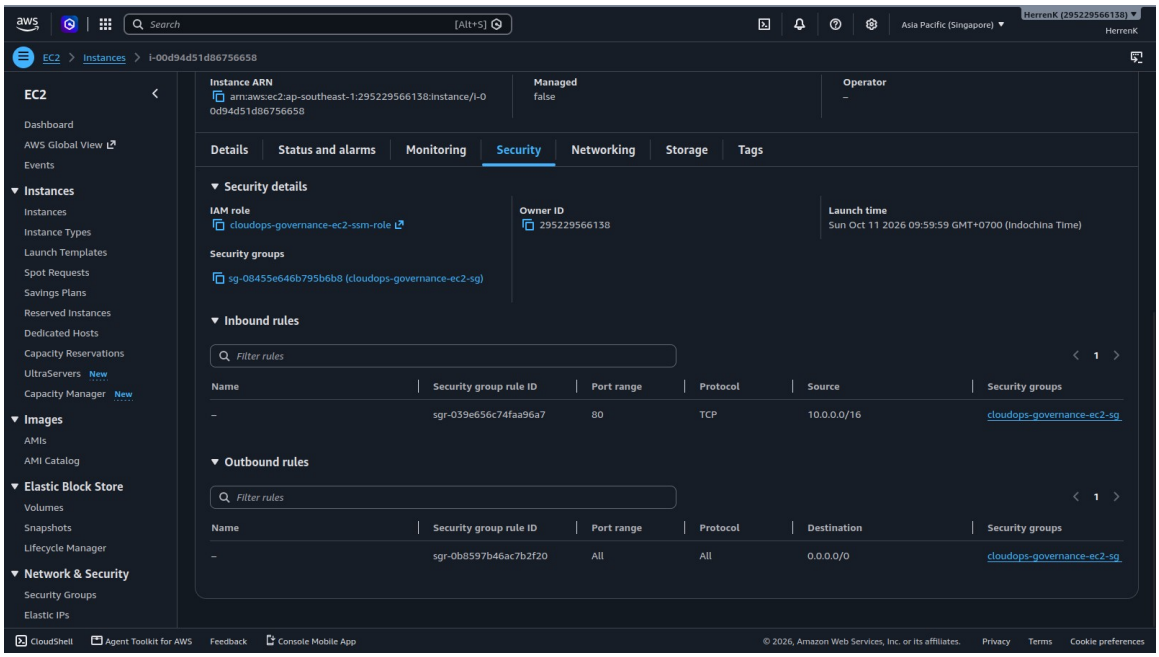
ภาพที่ *4:* รายละเอียด *Security Group* ของ *RDS* ยืนยันว่ายอมรับทราฟฟิกพอร์ต *5432* จาก *Security Group* ของ *EC2 (sg-08455e646b795b6b8)* เท่านั้น

ระยะที่ **4:** การรักษาความปลอดภัยเซิร์ฟเวอร์ **EC2** และการเข้าถึงแบบ **Zero-Trust**

เครื่องเซิร์ฟเวอร์ EC2 ถูกตรวจสอบสถานะการหยุดทำงาน และตรวจสอบนโยบายการตัดช่องทางการเชื่อมต่อผ่านพอร์ต SSH สาธารณะ



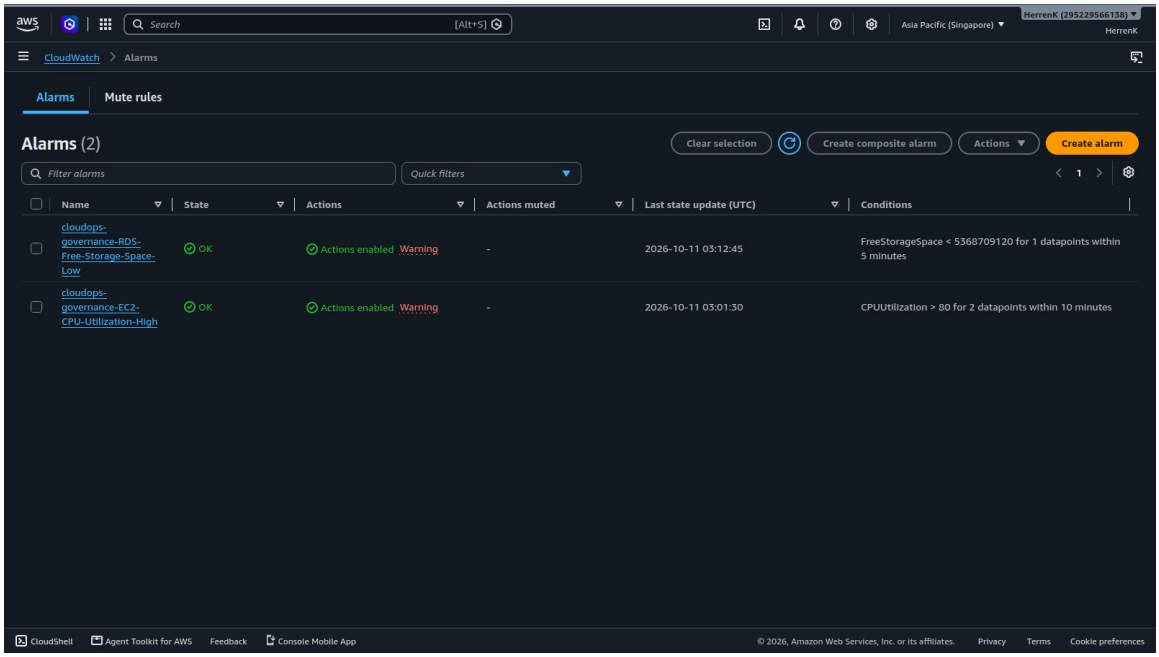
ภาพที่ *5:* หน้าจอ *Amazon EC2 Console* แสดงสถานะเครื่องเป็น *'Stopped'* อยู่ใน *Subnet* ส่วนตัว และผูกกับ *IAM Role* ของ *AWS Systems Manager (SSM)*



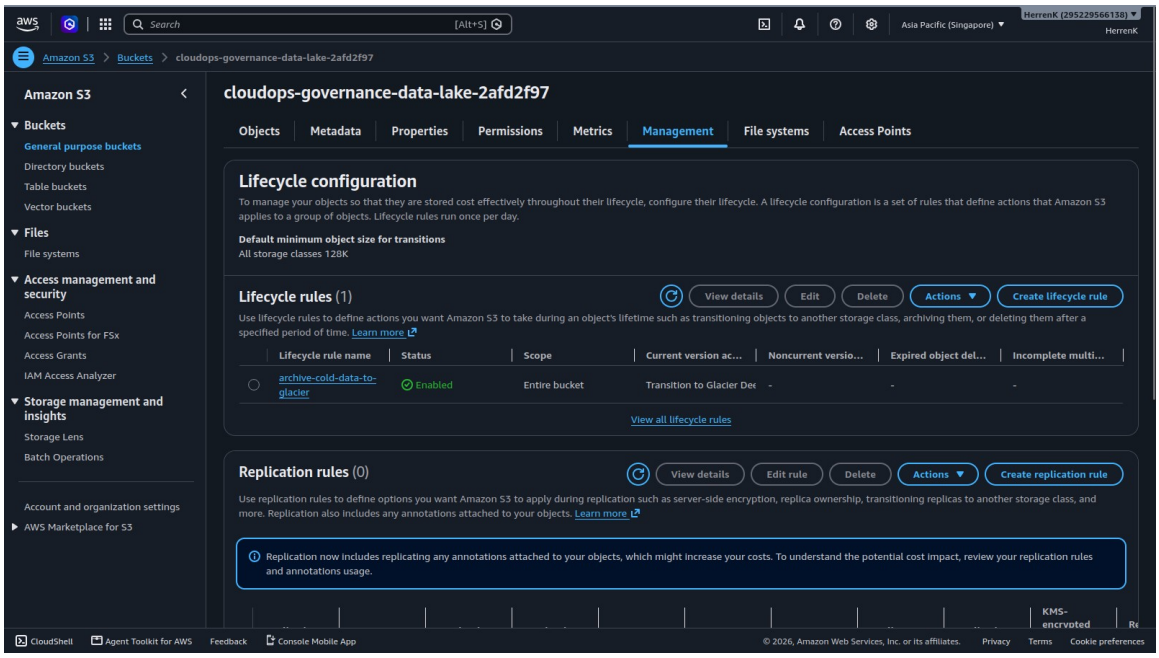
ภาพที่ 6: รายละเอียด *Inbound Rules* ของ *EC2* แสดงว่าเปิดเฉพาะพอร์ต *HTTP 80* ภายในวงแลน โดยไม่มีการเปิดพอร์ต *22 (SSH)* สู่สาธารณะแต่อย่างใด

ระยะที่ 5: ระบบเฝ้าระวัง **CloudWatch Alarms** และการลดต้นทุน **Storage** ด้วย **S3 Lifecycle**

ระบบเฝ้าระวังและนโยบายจัดเก็บข้อมูลระยะยาวได้รับการตรวจสอบความพร้อมใช้งาน



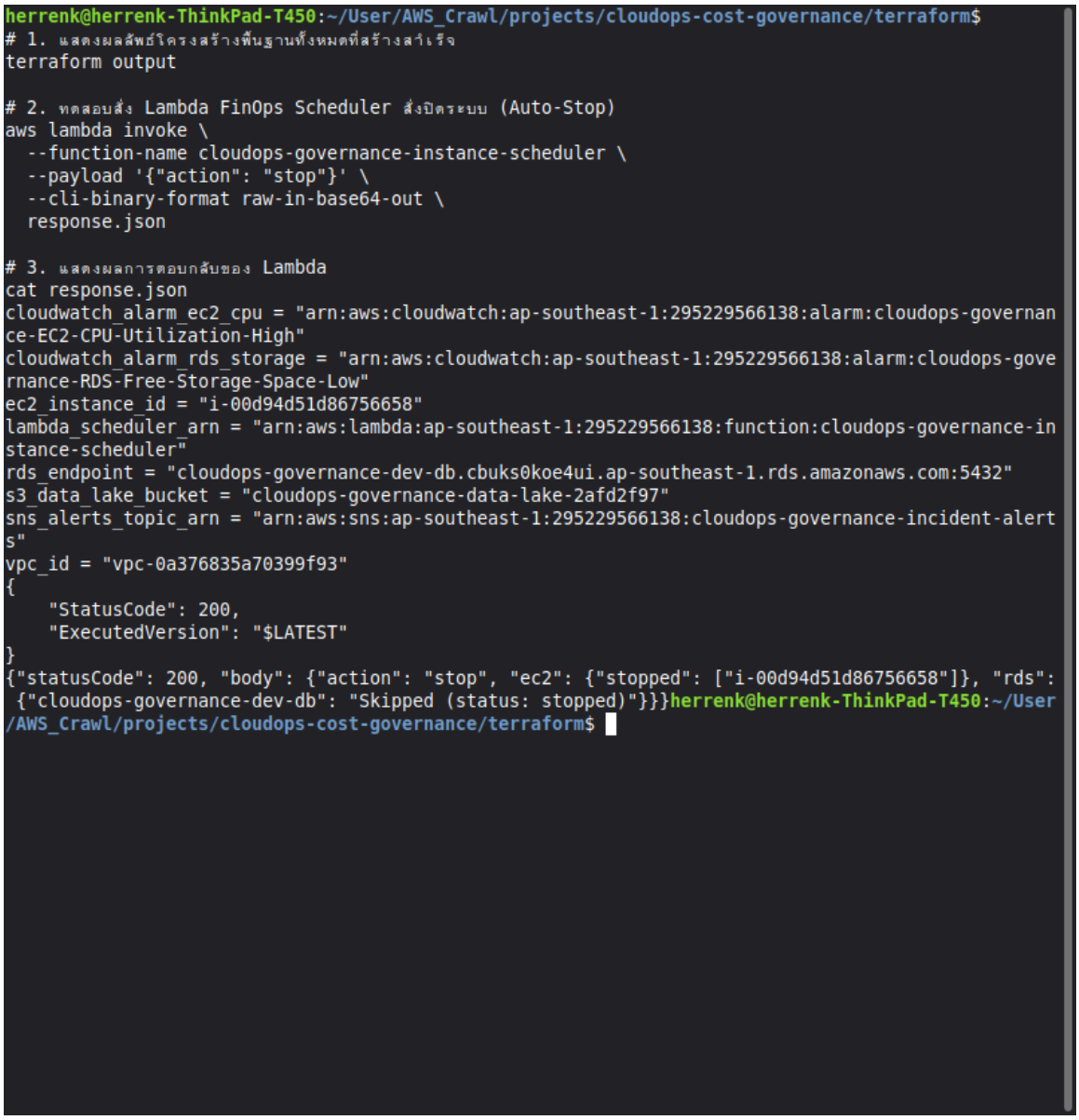
ภาพที่ 7: หน้าจอ *Amazon CloudWatch Console* แสดงสัญญาณเตือนทั้ง *2* ตัวอยู่ในสถานะสีเขียว *(OK)* พร้อมส่งการแจ้งเตือนเข้าสู่ *Amazon SNS* เมื่อเกิดเหตุผิดปกติ



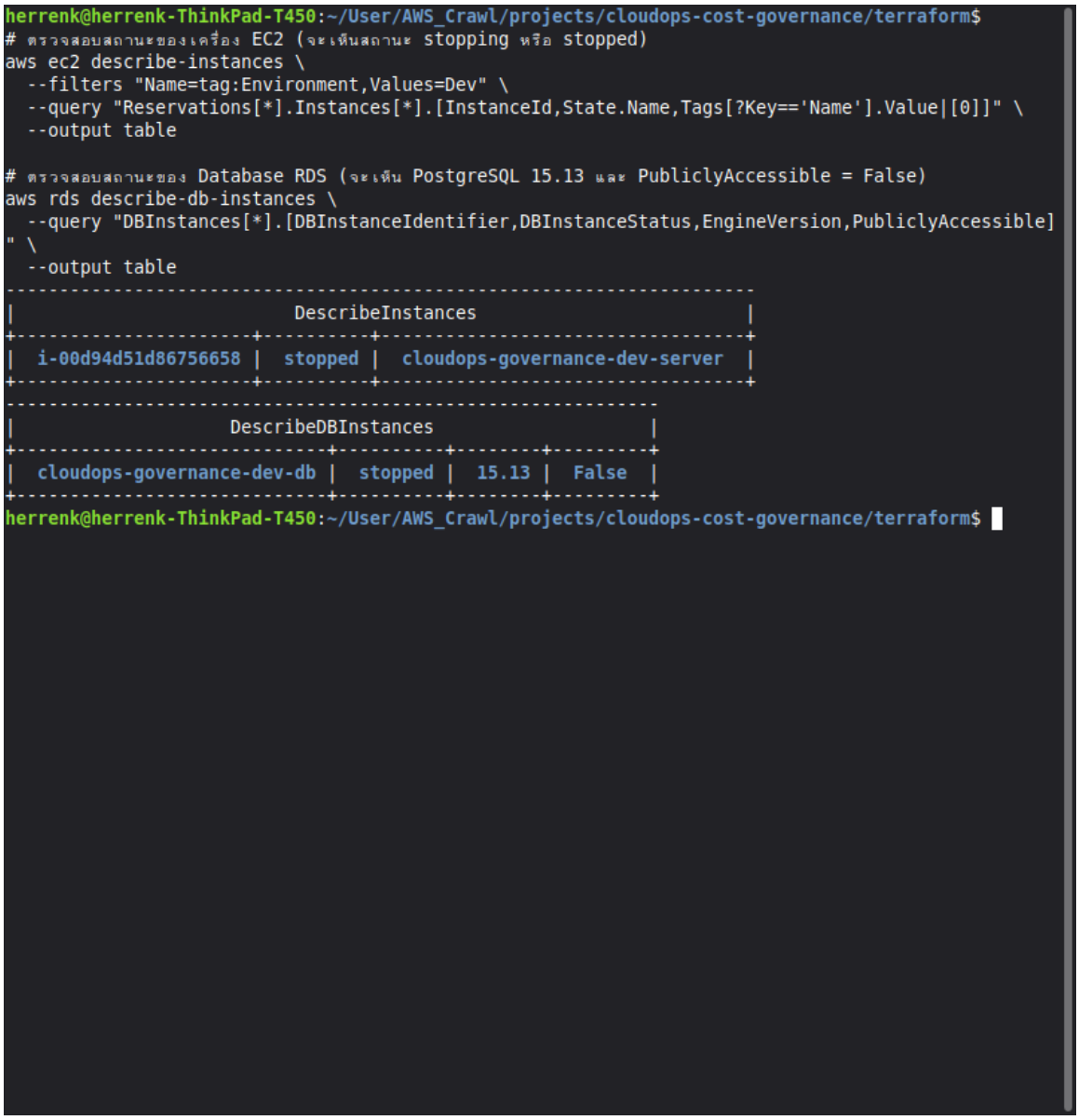
ภาพที่ 8: หน้าจอ *Amazon S3 Console* แสดงถัง *Data Lake* พร้อมกฎ *Lifecycle* ย้ายข้อมูลเก่าเกิน *90* วันไปยัง *Glacier Deep Archive* โดยอัตโนมัติ

ระยะที่ **6:** การตรวจสอบสถานะผ่าน **CLI** และการรันสคริปต์ตรวจความคุ้มค่า **FinOps**

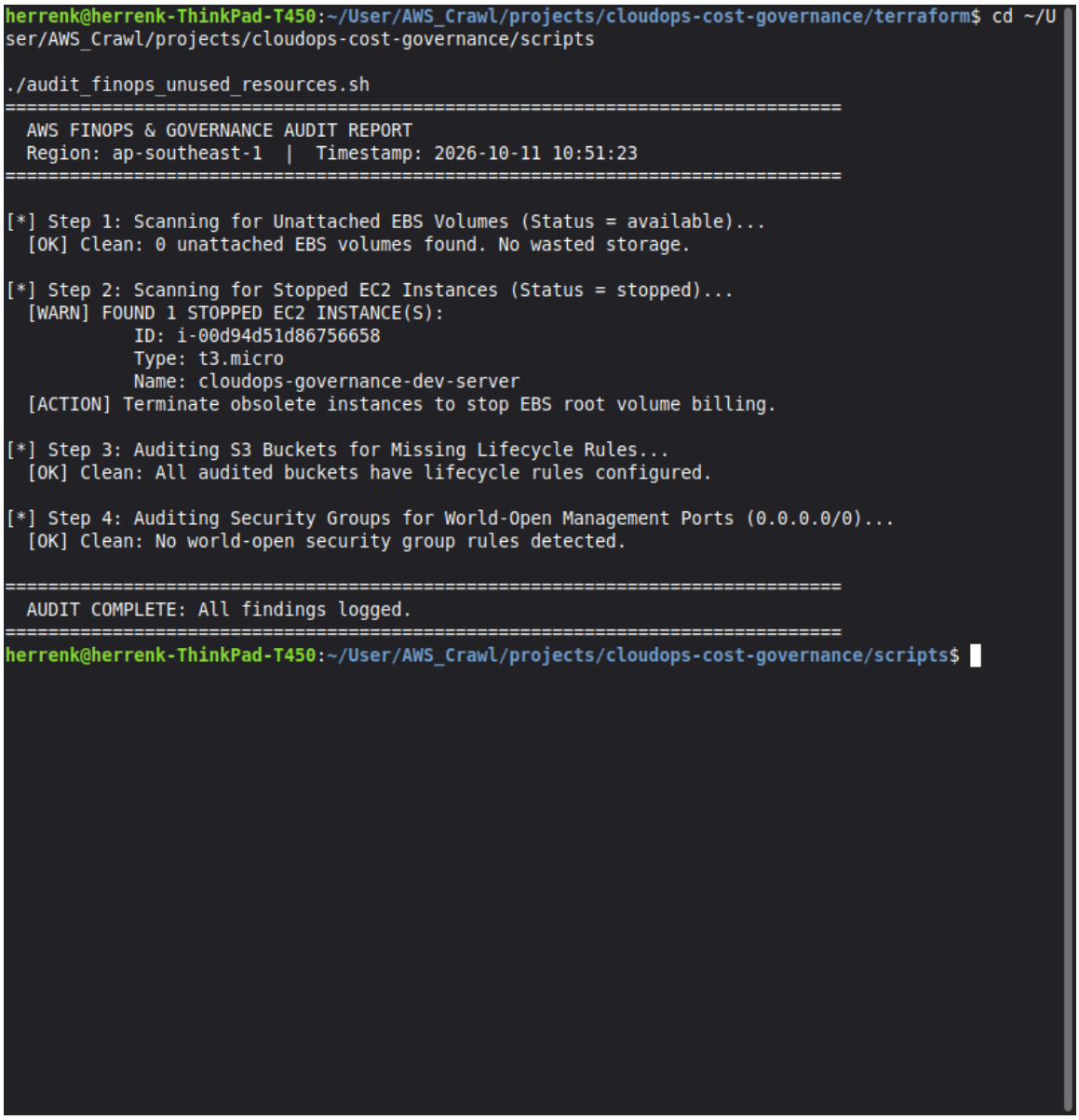
การสั่งงานผ่าน Terminal เพื่อตรวจสอบความถูกต้องของการประมวลผล สภาพ Idempotency และการทดสอบระบบเปิดเครื่องกลับมาใหม่



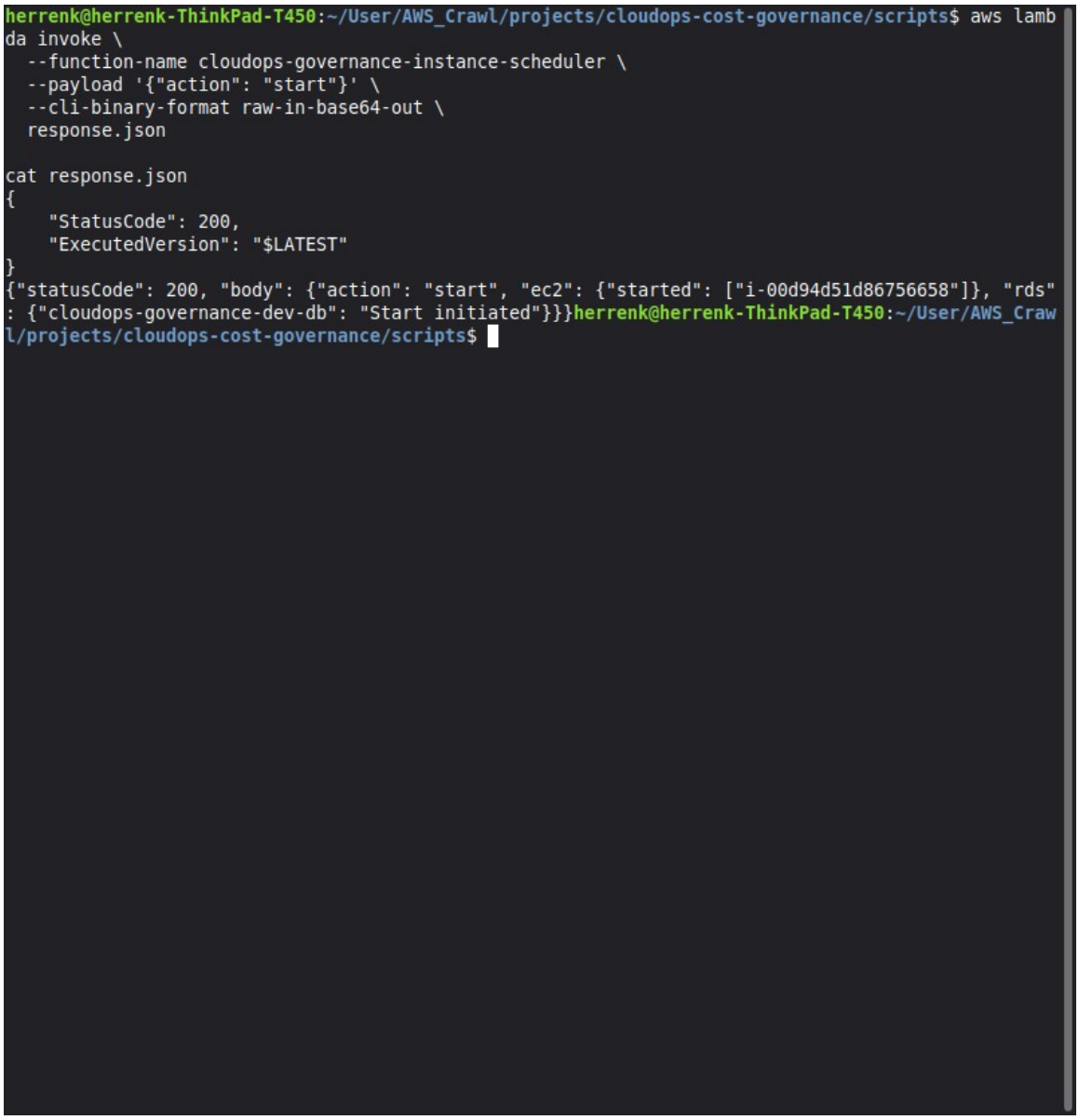
ภาพที่ *9:* การทดสอบคุณสมบัติ *Idempotency* ของ *Lambda:* เมื่อสั่งปิดซ้ำ ระบบตรวจพบว่าฐานข้อมูลหยุดทำงานอยู่แล้ว จึงข้ามการสั่งซ้ำได้อย่างปลอดภัย



ภาพที่ *10:* คำสั่ง *AWS CLI* แสดงตารางยืนยันสถานะ *EC2* เป็น *'stopped'* และ *RDS* เป็น *'stopped'* พร้อมระบุเวอร์ชัน *15.13* และ *PubliclyAccessible = False*



ภาพที่ *11:* ผลลัพธ์การรันสคริปต์ *audit_finops_unused_resources.sh* ครบ *4* ขั้นตอน ตรวจจับพบเครื่องที่หยุดทำงานเพื่อเตือนค่าใช้จ่าย *EBS* ได้อย่างแม่นยำ



ภาพที่ 12: การทดสอบคำสั่ง *action: start* เพื่อปลุกเครื่องเซิร์ฟเวอร์และฐานข้อมูลกลับมาพร้อมใช้งานสำหรับวันทำงานใหม่ *(Start initiated* สำเร็จ*)*

4. การวิเคราะห์ต้นทุนทางการเงินเชิงปริมาณ **(Financial & Cost Analysis)**

1. การเปรียบเทียบเวลาทำงานของระบบ: ในการเปิดเครื่องตลอด 24/7 เครื่องจะทำงาน 168 ชั่วโมงต่อสัปดาห์ แต่ด้วย FinOps EventBridge Scheduler เครื่องจะทำงานเฉพาะเวลา 08:00 - 19:00 น. วันจันทร์ถึงศุกร์ คิดเป็น 55 ชั่วโมงต่อสัปดาห์ ส่งผลให้ลดชั่วโมงการประมวลผลลงได้ทันที 67.26%

2. ความคุ้มค่าของพื้นที่จัดเก็บข้อมูล S3: ค่าจัดเก็บ S3 Standard อยู่ที่ประมาณ $0.023/GB/เดือน ขณะที่ S3 Glacier Deep Archive อยู่ที่เพียง $0.00099/GB/เดือน การย้ายข้อมูลเย็นที่เกิน 90 วันช่วยลดค่าจัดเก็บข้อมูลระยะยาวลงได้ถึง 95.70%

3. สรุปค่าใช้จ่ายจริงของโครงการ: ทรัพยากรทั้งหมด (EC2 t3.micro, RDS db.t3.micro, พื้นที่จัดเก็บ 20 GB gp3, EBS Volume 8 GB, และคำสั่ง Lambda) ถูกจำกัดให้อยู่ในโควตา AWS 12-Month Free Tier อย่างเคร่งครัด ทำให้ค่าใช้จ่ายจริงตลอดการทดสอบอยู่ที่ $0.00 สมบูรณ์แบบ

5. ประเด็นเด่นในการตอบสัมภาษณ์งาน **(Technical Interview Highlights)**

- **1.** การสร้างและทำลายระบบได้ทันที **(IaC Reproducibility):** 'โครงสร้างพื้นฐานทั้ง 35 รายการถูกสร้างด้วย Terraform แบบ 100% สามารถสร้างเสร็จใหม่ได้ภายใน 8 นาที และสั่งทำลายทิ้งได้ใน 5 นาทีด้วยคำสั่ง terraform destroy ช่วยลดความเสี่ยงเรื่อง Configuration Drift และการันตีว่าค่าใช้จ่ายในสภาพแวดล้อม Non-Production จะเป็นศูนย์เมื่อไม่ได้ใช้งานครับ'
- **2.** ความปลอดภัยแบบ **Zero-Trust:** 'เราไม่ได้พึ่งพาแค่ Firewall ภายนอก แต่ใช้แนวคิด Defense-in-Depth โดยทำ Security Group Chaining ให้ฐานข้อมูลคุยได้เฉพาะกับ EC2 ภายใน และปิดพอร์ต SSH 22 ทั้งหมด โดยเชื่อมต่อผ่าน AWS Systems Manager แทนเพื่อความปลอดภัยสูงสุดครับ'
- **3.** สถาปัตยกรรม **FinOps** อัจฉริยะ**:** 'ระบบ FinOps ใช้การค้นหาทรัพยากรผ่าน Tag Environment=Dev แบบไดนามิก ทำให้ฟังก์ชัน Lambda สามารถควบคุมการเปิด-ปิดเครื่องได้โดยไม่ต้องระบุ Instance ID แบบฮาร์ดโค้ด รองรับการขยายตัวของระบบในอนาคตได้อย่างยืดหยุ่นครับ'
- **4.** การออกแบบโค้ดที่ทนทานต่อข้อผิดพลาด**:** 'ตัวฟังก์ชัน Lambda มีการตรวจสอบสถานะจริงของเครื่องก่อนสั่งการ (Idempotent Design) เพื่อป้องกัน Error ในกรณีที่เครื่องอยู่ในสถานะ Stopping หรือ Stopped อยู่แล้วครับ'

6. บทสรุปและการทำลายระบบเพื่อความสะอาด **($0 Teardown Verification)**

ทรัพยากรทั้ง 35 รายการในโครงการนี้ผ่านการทดสอบด้านสถาปัตยกรรม ความปลอดภัย และการบริหารต้นทุนครบถ้วน หลังจากบันทึกหลักฐานการทำงานเรียบร้อยแล้ว ได้ทำการสั่งทำลายทรัพยากรทั้งหมดผ่านคำสั่ง terraform destroy -auto-approve ซึ่งระบบได้ลบทั้ง 35 รายการออกอย่างหมดจดภายในเวลาไม่ถึง 6 นาที (โดย RDS มีการตั้งค่า skip_final_snapshot = true และ S3 มีการตั้งค่า force_destroy = true เพื่อรับประกันการลบที่สะอาดและไม่มีภาระค่าใช้จ่ายหลงเหลือบน AWS อีกต่อไป)